

Log-Analyzer System

Advanced Data Processing with Python

A compact, high-performance console application for parsing, analyzing, and exporting system logs. Built with modular architecture to support maintainable development and scalable processing of large files.



Team Members

Yerassyl Irangait — Integration & Git Flow

Ramazan Aljanov — Data modeling & validation

Ayazhan Yeslambek — Regex & memory optimization

Bayan Daulettkyzy — Filtering & statistics



Project Context

University project demonstrating production-grade Python: emphasis on correctness, performance, and testability.

```

py > ...
import os
from utils.parser import read_logs
from services.analyzer import filter_logs_by_status, count_log_stat
from utils.file_handler import export_to_json

```

```

def main():
    """Main entry point for the Log Analyzer application."""
    print("--- Welcome to the Log Analyzer System ---")

    # Define paths
    log_file_path = "data/sample_logs.txt"
    export_file_path = "data/status_counts.json"

    # Check if data file exists
    if not os.path.exists(log_file_path):
        print(f"Error: Could not find the log file at '{log_file_path}'")
        return

    # 1. Load data using the generator from Participant 2
    print("Loading logs...")
    logs = list(read_logs(log_file_path))
    print(f"Successfully loaded {len(logs)} valid log entries.\n")

    # 2. Start the interactive menu
    while True:
        print("\n=== Main Menu ===")
        print("1. Filter logs by status (e.g., ERROR, INFO)")
        print("2. Count all log statuses")
        print("3. Export status counts to JSON")
        print("4. Exit")

        choice = input("Enter your choice (1-4): ")

        if choice == '1':
            status = input("Enter status to filter: ").strip().upper()
            filtered = filter_logs_by_status(logs, status)

            print(f"\nFound {len(filtered)} logs with status '{status}'")
            # Show up to 5 logs so we don't flood the console
            for i, log in enumerate(filtered[:5]):
                print(f"{i+1}. {log}")

```

Project Overview

Goal: deliver a robust console program to parse, analyze, and filter system logs efficiently. Key constraints: process files larger than memory, maintain low latency for common queries, and provide clear export formats (JSON).

Modular Architecture

Separation into Parser, Models, Analyzer and Tests – enables parallel development and isolated unit testing.

Non-functional Goals

Memory efficiency, predictable performance, and clear error handling for malformed entries.

The Team & Roles



Yerassyl Irangait

Coordinates integration, defines Git branching strategy, creating interface, managing the process



Ramazan Aljanov

Designs domain models, validation rules, and enforces encapsulation and clear APIs between modules.



Ayazhan Yeslambek

Creates efficient regex patterns, implements streaming parsers using generators, and optimizes memory use.



Bayan Dauletkyzy

Implements filters, aggregation logic, and statistical summaries; validates results against expected distributions.



Role boundaries kept narrow to simplify unit testing and code ownership.

Core Technology — OOP & Models

We model log records as typed objects. `LogEntry` provides shared behavior; specialized subclasses (e.g., `InfoLog`, `ErrorLog`) encapsulate level-specific parsing and metadata.

1

Inheritance

Shared parsing utilities and formatted output live in the base class to avoid duplication.

2

Encapsulation

Use `@property` getters to compute derived fields and prevent external mutation.

3

Validation

Constructor-level validation raises descriptive exceptions for malformed timestamps or missing fields.

```
if status not in valid_statuses:
    raise ValueError(f"Invalid log status:
    {status}. Must be one of {valid_statuses}")
```

Core Technology — High-Performance Parsing

Parsing engine focuses on two pillars: robust Regular Expressions for extraction, and Generators (yield) for lazy, memory-efficient file processing. This enables streaming through very large logs without loading them fully into memory.

```
import re
from models.log_entry import LogEntry

def read_logs(file_path):
    pattern = re.compile(r"^(\d{4}-\d{2}-\d{2} \d{2}:\d{2}:\d{2})\s+([A-Z]+)\s+(.+)$")
    ...
```

1 Regex Strategy

Precompile patterns, use non-capturing groups when appropriate, and apply anchored matches to reduce backtracking.

2 Generators & Lazy IO

File readers yield validated log entries; downstream processors consume iterators with minimal buffering.

Core Technology — Advanced Analytics

We leverage higher-order functions to compose lightweight, expressive processing pipelines. Using `filter()`, `map()`, and generator expressions avoids redundant loops and clarifies intent.

Filter & Transform

Compose filters to select by level, time range, or regex groups; map transforms entries into aggregates or exportable dicts.

Aggregation

Use `itertools` and `collections` (`Counter`, `defaultdict`) to compute counts and histograms in a single pass.

```
def filter_logs_by_status(logs, status):
    return list(filter(lambda log: log.status == status, logs))

def count_log_statuses(logs):
    statuses = list(map(lambda log: log.status, logs))
    status_counts = {}
    for current_status in statuses:
        status_counts[current_status] = status_counts.get(current_status, 0) + 1
    return status_counts
```

Quality Assurance — Testing Strategy

Each module includes a dedicated unittest suite. Tests cover parsing edge cases, model validation, filter correctness, and CLI behaviors. Integration tests run in CI to exercise typical end-to-end flows on representative sample files.

Unit Tests

Parser, models, and analyzer modules have isolated test cases with synthetic fixtures and property-based checks where appropriate.

Integration / CI

End-to-end tests validate streaming performance and JSON export integrity; included with time and memory budgets.

📁 Mocks and temporary file fixtures are used to keep tests deterministic and fast.

Interface & UX — Interactive Console Menu

The CLI provides a guided menu: Load file → Apply filters → View counts/summary → Export. Designed for keyboard-first workflows with clear prompts, progress indicators, and JSON export for downstream tools.

 UX choices prioritize simplicity. The CLI provides clear prompts and uses input validation to prevent crashes from invalid data.

01

Load

Path validation and memory-safe data loading without overloading RAM (using `yield` generators).

03

Count & Inspect

Fast aggregation of log counts. To avoid console flooding, only the first 5 matching records are displayed.

02

Filter

Interactive filtering where the user types the desired status (e.g., ERROR, INFO) directly in the console.

04

Export

Conveniently exporting the final status counts into a standard JSON file for further use.

Summary & Outcomes

Outcome: A modular, well-tested Log-Analyzer that streams massive log files with minimal memory footprint, precise parsing, and concise analytical primitives. System supports extensibility for new log formats and custom filters.

Performance

Streaming parsers and generator-based pipelines reduce peak memory to near-constant for typical workloads.

Maintainability

Clear separation of concerns, comprehensive unit tests, and documented APIs allow safe future enhancements.

Reproducibility

CLI supports configuration files and logging for repeatable analyses and automation.